Universidade Leiria • Oeste
Escola de Tecnologia e Gestão

Information Systems Integration

Part 2 – Agnostic data representation
Introducing XML and JSON

Introducing XML

- **The need: a motivating example** [1]
  - An online/mobile banking system:
    - Allows customers to deposit money on their account from another bank
    - Front-end mobile application which integrates with back-end of the national fund transfers authority
      - Simplification: assume remote transfer takes only the person's name and the dollar amount

Introducing XML

- The need: a motivating example
  - **Firstly: Connection?**
    - TCP/IP, why not? It is the most ubiquitous communication protocol…
      - Support from ‘all‘ operating systems and programming languages…

Introducing XML

The need: a motivating example

```
String hostName = "www.sibs.pt";                                    [1]
int port = 80;
IPHostEntry hostInfo = Dns.GetHostByName(hostName);
IPAddress address = hostInfo.AddressList[0];
IPEndPoint endpoint = new IPEndPoint(address, port);
Socket socket = new Socket(address.AddressFamily, SocketType.Stream,
ProtocolType.Tcp);
socket.Connect(endpoint);
DataOutputStream d_o = new DataOutputStream(socket.getOutputStream());
d_o.writeInt(1000);
String str="Joe";
byte[] data=str.getBytes("UTF-8");
d_o.write(data);
socket.Close();
```

7 bytes sent. “So simple!”
How can integration be so difficult?

Introducing XML

- The need: a motivating example
  - Problems:
    - **1. Platform independence? (binary data was used!)**
      - Size of int!

.NET int is 32 bit large but in other OSs and programming languages…

- A 64-bit int based system will interpret customer name as dollar amount, right?

Introducing XML

- The need: a motivating example
  - Problems:
    - 1. Platform independence works only for byte streams
      - **In-memory bit order? Little-endian vs big-endian**
        - Numbers: highest byte first vs lowest byte first

232 3 0 0 **vs** 0 0 3 232

Introducing XML

- The need: a motivating example
  - Problems:
    - **2. Suppose function/API is moved to different computer on a different domain**
      - What if machine fails and new machine setup?

Code is too tied to "www.sibs.pt"

Introducing XML

- The need: a motivating example
  - Problems:
    - **3. Temporal dependencies**
      - TCP/IP is connection oriented
        - Some messages exchanged just to stablish connection
        - Sender and receiver must exist at the same time!

If there are networks problems, data is not sent. Too much tightly coupled!

Introducing XML

- The need: a motivating example
  - Problems:
    - **4. Data format (add new field)**
      - 4 bytes (amount) + sequence of characters (name)
        - Suppose we need to add a new data field to the message such as currency name…

Both sender and receiver must be modified ☹

Introducing XML

- The need: a motivating example
  - From now on, we will focus on **data format problem**

Introducing XML

- What Is XML
  - XML stands for E**x**tensible **M**arkup **L**anguage
  - Aim: **data representation standard**
    - REM: binary file => bound to application (stream of bits)!
      - Opening it with text editor… “rubbish”
      - Format must be known!
      - However: less space and less bandwidth!

Introducing XML

- What Is XML
  - What about text files?
    - Human and machine readable! ☺
      - Each group of bits represents a char (known charset)
      - **What about encodings?**
        - ANSI, UTF-8, etc.
        - Let's assume first line stores it. Great!
      - However, it uses more space (disk/bandwidth)

?

Introducing XML

- What Is XML
  - What about text files?
    - **What about data semantic?**
      - Labrador;12;33.56 (What is this?)



Introducing XML

- What Is XML
  - What about text files?
    - **Metadata/markup is the answer!**
      - “raça”:”labrador”,”idade”:12,”peso”:33.56

Both JSON and XML are "self-describing" (human readable)

Both JSON and XML are hierarchical (values within values)

Both JSON and XML can be parsed

Introducing XML

- What Is XML
  - **The markup is the answer!**
    - Surrounding text with **extra** information
    - However,…
      - How to differentiate metadata from text
      - What metadata is allowed
      - Metadata must be created **as needed**!!
        - Standard Generalized Markup Language (SGML/ISO)
          - Example: HTML (header, body, tags, etc.)

Introducing XML

- What Is XML
  - **Ok, SGML is very complex and static tag set**
    - Task force scheduled for subset of SGML
    - 1998: XML was published (and approved) by W3C
      - E**x**tensible **M**arkup **L**anguage
      - Does not dictate the overall format like SGML
      - Just a **few** rules! ☺

Introducing XML

What Is XML

Example: list of application users

Joe Fawcett
Danny Ayers
Catherine Middleton

No metadata!

XML version

Attributes

Elements

```
<applicationUsers>
<user firstName="Joe" lastName="Fawcett" />
<user firstName="Danny" lastName="Ayers" />
<user firstName="Catherine" lastName="Middleton" />
</applicationUsers>
```

Rem: file size increasing!

XML

- **XML** is markup-language for data representation
  - Used for encoding documents in machine-readable form
  - Text-based format, consists of tags, attributes and content
  - Provide data and meta-data at the same time

```
<?xml version="1.0"?>
<library>
  <book><title>HTML 5</title><author>Jon Doo</author></book>
  <book><title>Web Book</title><author>Microsoft</author></book>
  <book><title>UML 2.0</title><author>Steve Job</author></book>
</library>
```



JSON

- JSON (**J**ava**S**cript **O**bject **N**otation)
  - Standard for representing data structures and associative arrays
  - Lightweight text-based open standard
  - Derived from the JavaScript language

keys and values

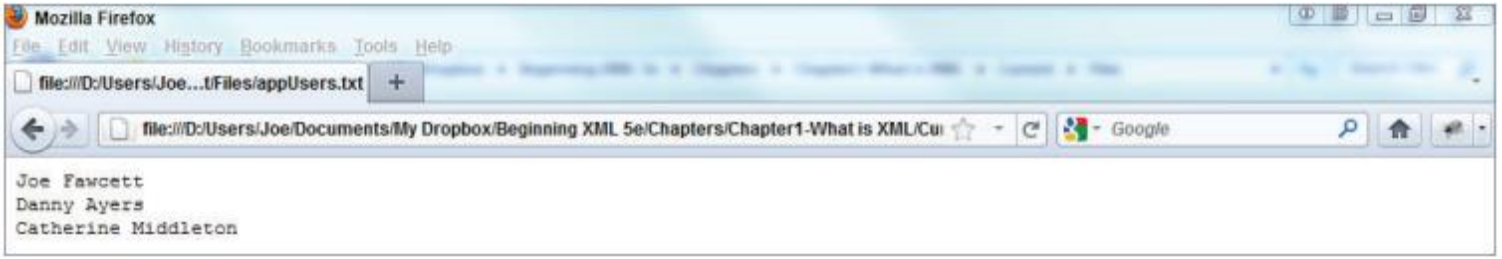
```
{
  "firstName": "John", "lastName": "Smith", "age": 25,
  "address": { "streetAddress": "17 Tintyava Str.",
               "city": "Sofia", "postalCode": "1113" },
  "phoneNumber": [ { "type": "home", "number": "212 555-1234"},
                   { "type": "fax", "number": "646 555-4567" }]
},
{ "firstName": "Bay", "lastName": "Ivan", "age": 79 }
```

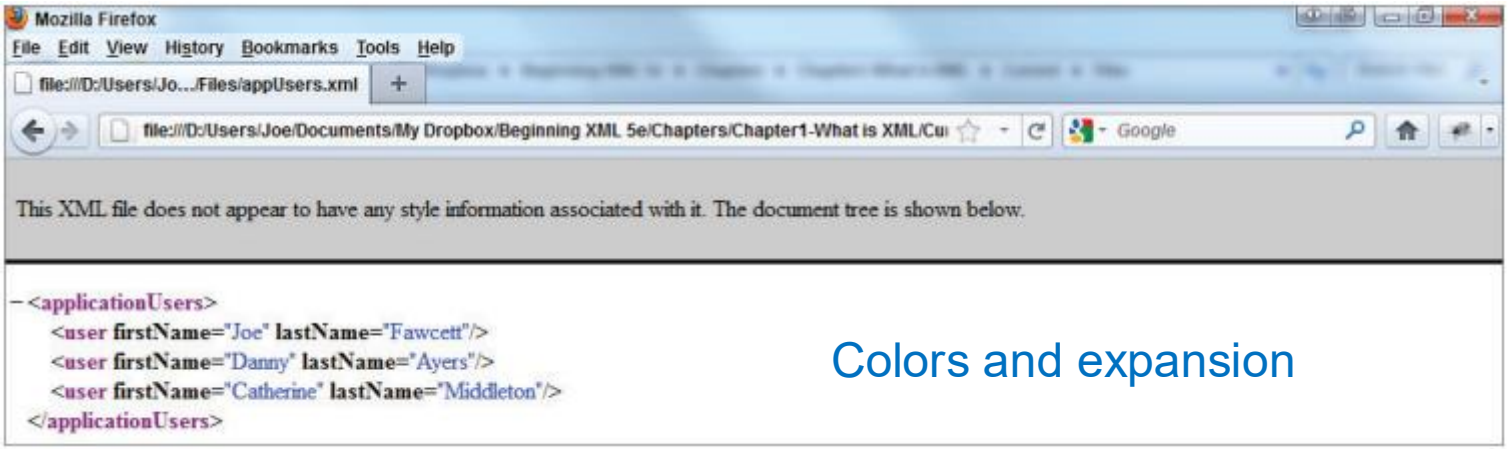


Introducing XML

What Is XML

Save previous data into .txt file and into .xml file and use the browser to open files…





Colors and expansion

Introducing XML

- What Is XML
  - **Others XML advantages**
    - It separates data from presentation (unlike HTML)
    - Simple rules:
      - Ex: Attribute's values within quotes
        - `<user lastName="Fawcett"/>`

Introducing XML

- What Is XML
  - Others XML advantages
    - **Extensibility**
      - E.g., easy to add middle name
        - `<user firstName=”Danny” middleName=”John” lastName=”Ayers” />`



What about existing applications?

Introducing XML

- What Is XML
  - Others XML advantages
    - **Easy hierarchical data representation**
      - Ex: file system

```
<folder name="C:\">
  <folder name="Program Files">
    <folder name="Microsoft">
    </folder>
    <folder name="Mozilla">
    </folder>
  </folder>
  <folder name="Windows>
    <folder name="System32">
    </folder>
  </folder>
```

Introducing XML

- What Is XML
  - Others XML advantages
    - **Interoperability**
      - Exchanging data among different applications

Word > 2003 saves documents in XML format



Interoperability with LibreOffice for example

Suppose now .docx gets corrupted…

?

Introducing XML

- What Is XML
  - **XML usage scenarios**
    - Configuration Files (hierarchies and processing)
    - Web Services (serialization and return values)
    - Web content (XHTML, lightweight)
    - Documents (author, versions, etc. in one place)
    - Databases (column type)
    - Image representation (SVG format)
    - Business interoperability (ex: retail)

Introducing XML

More details on lab classes

XML Technologies

- XML Parsers (loading XML into the RAM)
  - Do not use substring, strlength, etc…!
  - Some commonly used parsers:
    - **.NET System.Xml.XmlDocument**
    - Saxon (tools to parse, transform and query XML)
      - Java and .NET (Free edition)
    - Java built-in parser
    - Xerces (Apache Software Foundation, Java based)

Introducing XML

- XML Technologies
  - **Document Object Model (DOM)**
    - Model to keep XML data in-memory after parsing
      - Tree based approach
      - Memory consumption: 1MB file ~= 5MB in-memory
    - The standard also applies to HTML
    - Widely supported (browsers, JQuery, etc.)

More details on lab classes

Introducing XML

More details on lab classes

- XML Technologies
  - **XML Schemas (XSD)**
    - Specify XML document structure and data allowed
      - XML schemas are XML based and easier to use
        - **Class** – Objects vs **Schema type** – XML instances

An XML document with correct syntax is called "**Well Formed**"

An XML document validated against a XSD is both "Well Formed" and "**Valid**"

XML Schema=XML Schema Definition (XSD)
(Validation)

Introducing XML

- XML Technologies
  - **XML Namespaces**
    - XML Namespaces were added to the XML later
      - Work as namespaces in OOP programming languages
  - **XPATH**
    - Gives XML the path notation (used in lots of XML tools)
      - **/applicationUsers/user** - to select all users
        /applicationUsers/user/@name – the name attribute of all users
        //user[@password] – select all users elements with password attribute

More details on lab classes



Introducing XML

- XML Technologies
  - **XSLT - Extensible Stylesheet Language Transformations**
    - Transforms one format into another
      - Example: transforms XML into HTML (server or client side)

```
...
<xsl:template match="user">
   <tr>
      <td>
         <xsl:value-of select="@firstName"/>
      </td>
      <td>
         <xsl:value-of select="@lastName"/>
      </td>
  </tr>
</xsl:template>
</xsl:stylesheet>
```



Introducing XML

- XML Technologies
  - **XQuery – does what XSLT does and more**
    - Does not follow the XML format (is like an embedded programming language)
    - Suitable for processing large collections (e.g., XML on databases)

```
...
{for $user in doc("appUsers.xml")/applicationUsers/user
return <tr><td>{data($user/@firstName)}</td>
<td>{data($user/@lastName)}</td></tr>}
  </tbody>
</table>
</body>
</html>
```

References/Bibliography

[1] Gregor Hohpe, and Bobby Woolf, “Enterprise Integration Patterns: Designing Building, and Deploying Messaging Solutions”, Pearson Education, Inc., 2003

[2] Joe Fawcet, Liam Quin, and Danny Ayers, “Beginning XML”, fifth edition, John Wiley & Sons, Inc, 2012.